

Version 1.2

January 2026



DATA STRUCTURES

MODULE 5 - HASHMAP

Author:

MUHAMMAD ILHAM PERDANA. S.Tr.T., M.T.

AHMAD NUR MU'MININ

WAHYU ANDIKA

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
REFERENCES.....	3
CONTENTS.....	4
1. HashMap.....	4
1.1 Definition of HashMap.....	4
1.3 Hashing Concept.....	5
2. Collision Handling.....	6
2.1 What is Collision?.....	6
2.3 Open Addressing (Linear Probing).....	7
3. Basic Hashman Operations.....	8
3.1 Adding Data.....	8
3.2 Retrieving Data.....	9
3.3 Removing Data.....	10
3.4 Checking Data Existence.....	11
3.5 Other Useful HashMap Methods.....	12
4. Iteration & Data Traversal.....	15
4.1 Iteration Using entrySet().....	16
CODELAB.....	17
Exercise.....	17
DEMO.....	20
TASK 1.....	20
TASK 2.....	21
Assesment.....	23
Grading Rubric.....	23
Grading Scale.....	23



INTRODUCTION

OBJECTIVES

1. Students are able to understand and explain the fundamental concepts of Hashing and Hashmap

MODULE TARGETS

1. Students are able to understand Data Structure HashMaps

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّٰهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. Java Development Kit
4. Java Runtime Environment
5. IDE (Intellij IDEA, Eclipse, Netbeans, etc.)
6. Strong motivation and commitment to learning

KEYWORDS

Hashmap, Hashing, Collision, Iteration, Data Traversal

REFERENCES

Note: These references may contain slight variations in explanation or examples. Focus on understanding the core concepts and apply them to the case study provided in this module.



CONTENTS

1. HashMap

1.1 Definition of HashMap

In everyday life, a HashMap can be compared to a lost-and-found counter at a railway station. Each item handed in is tagged with a unique identifier, such as a claim number. When someone comes to retrieve an item, the staff does not search through all stored items one by one. Instead, they simply look up the item using the claim number and immediately retrieve the corresponding object. In this analogy, the claim number acts as the key, and the stored item details act as the value, illustrating how HashMap enables fast and direct access to data using a unique identifier.

Key	Value
Nama →	Zaidun Andy Ahmad
NIM →	202510370311003
IPK →	3.75
1 →	Minggu
2 →	Senin

HashMap Concept: Each unique Key identifies and accesses its corresponding Value, similar to an ID system.

Figure 1. Hashmap Concept

1.2 Characteristics of Hashmap

The characteristics of a HashMap are similar to a **digital reservation board at a train station**. The board does not list bookings in the order they were made, but instead organizes them internally so that staff can instantly retrieve a reservation by entering the booking code. Even if two passengers share the same name, their booking codes remain unique. This reflects how HashMap enforces unique keys while allowing duplicate values, and why the order of stored data is not guaranteed or important in such systems.



Characteristic	Description
Fast Access	<i>put, get, remove</i> work in average $O(1)$ time
Unordered	Data order is not guaranteed
Unique Keys	Keys must be unique
Duplicate Values	Values may be duplicated
Null Support	One `null` key, multiple `null` values allowed
Not Thread-Safe	Not safe for multi-thread without synchronization

1.3 Hashing Concept

Hashing can be analogized to an **automated sorting system for ticket documents**. When a booking code is scanned, the system applies a rule to determine which storage drawer the document should go into. Staff do not manually decide the drawer; the system calculates it automatically based on the code. Similarly, in HashMap, the hashing function converts a key into a numeric value that determines where the data is stored internally, allowing fast and efficient retrieval without manual searching.

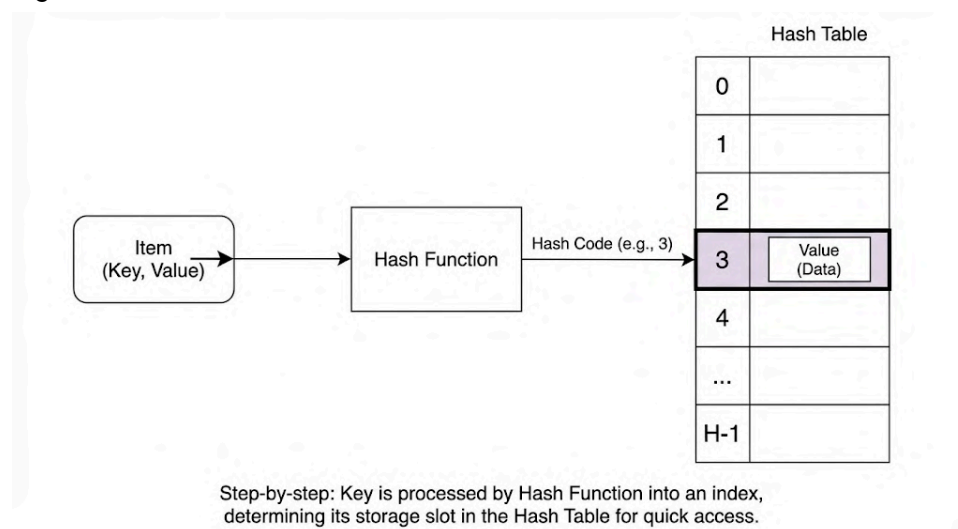


Figure 2. Hashing Concept



2. Collision Handling

2.1 What is Collision?

A collision in HashMap is similar to a situation where **two different passengers are assigned the same locker number by an automated system**. Even though their identities are different, the system accidentally maps them to the same location. This does not mean the system fails; instead, it must apply a strategy to handle both passengers' belongings safely. In HashMap, collision handling ensures that multiple keys mapped to the same bucket can still coexist without losing data.

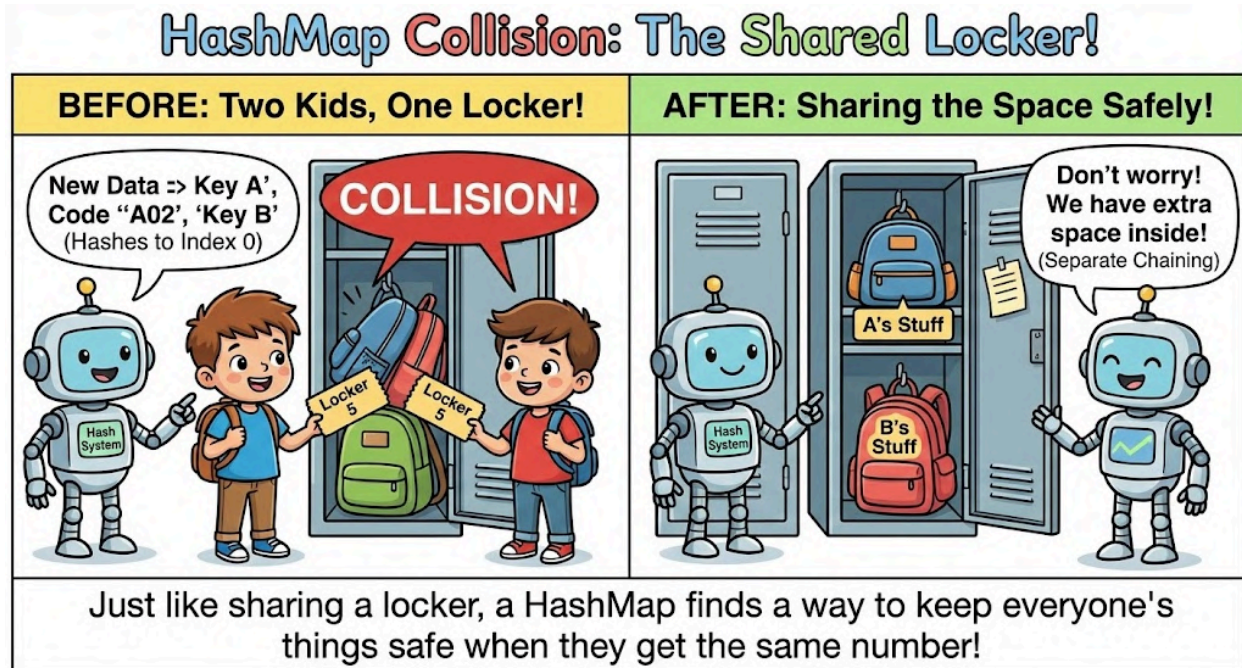


Figure 3. Collision

2.2 Separate Chaining

Separate chaining can be compared to **placing multiple documents in a single folder using dividers**. If several booking records are assigned to the same storage location, the system does not overwrite existing data. Instead, it stores all records in a linked sequence within the same folder. This allows staff to browse through the folder and find the correct document when needed. Similarly, HashMap uses linked lists to store multiple entries in the same bucket when collisions occur.



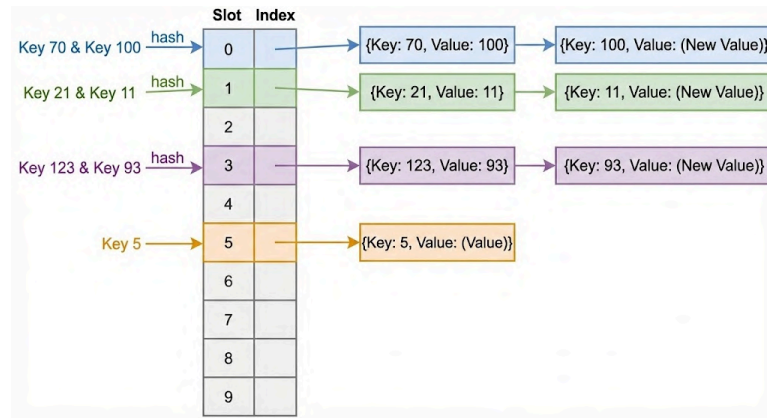


Figure 4. Separate Chaining

2.3 Open Addressing (Linear Probing)

Open addressing with linear probing resembles a **seat reassignment process on a train**. If a passenger's assigned seat is already occupied, the conductor checks the next available seat until an empty one is found. The passenger is then seated there instead of the original seat. In HashMap, when the target bucket is occupied, the system searches for the next available bucket to store the data, ensuring that every entry still has a place.

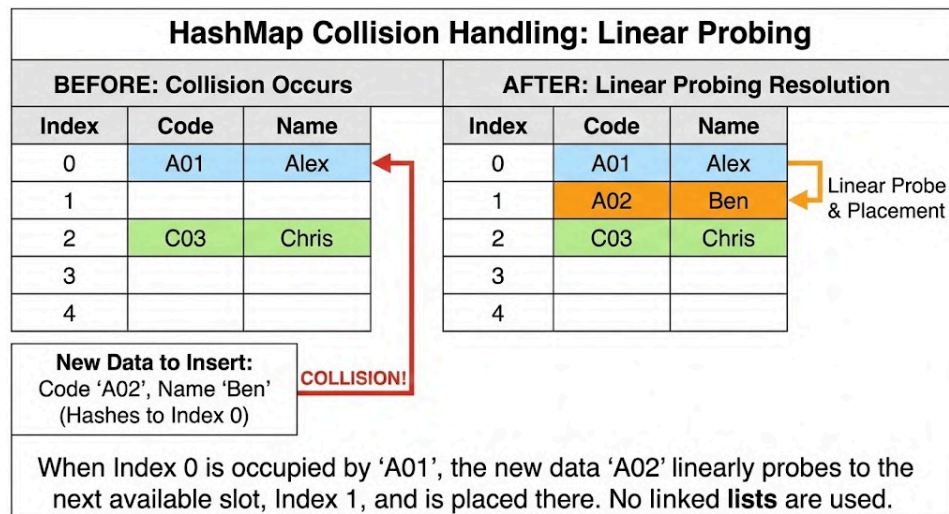


Figure 5. Linear Probing



3. Basic Hashman Operations

Create a Hashmap with the code below:

```
import java.util.HashMap;

public class ContohHashMap {
    public static void main(String[] args) {

        // Key berupa Integer, Value berupa String
        HashMap<Integer, String> days = new HashMap<>();

        // Key berupa String, Value berupa String
        HashMap<String, String> dataUser = new HashMap<String, String>();

        // Membuat HashMap dengan kapasitas awal (opsional)
        // Berguna untuk mengurangi proses rehashing jika data besar
        HashMap<String, Double> hargaBarang = new HashMap<>(50);

    }
}
```

3.1 Adding Data

Adding data to a HashMap using `put()` or `replace()` is similar to **registering or updating passenger information at a ticket counter**. When a new booking code is entered, the system creates a new record. If the booking code already exists, the old information is replaced with updated details, such as a schedule change or passenger name correction. This mirrors how HashMap either inserts new data or updates existing values based on the key.



```
import java.util.HashMap;

public class OperasiPutReplace {
    public static void main(String[] args) {

        HashMap<Integer, String> days = new HashMap<>();

        // Menambahkan data menggunakan put()
        days.put(1, "Minggu");
        days.put(2, "Senin");
        days.put(3, "Selasa");

        System.out.println("Isi awal: " + days);

        // Mengganti nilai dengan put()
        String nilaiLama = days.put(1, "Ahad");
        System.out.println("Setelah put(1, \"Ahad\"): " + days);
        System.out.println("Nilai lama key 1: " + nilaiLama);

        // Menambahkan key baru
        days.put(4, "Rabu");
        System.out.println("Setelah put(4, \"Rabu\"): " + days);

        // Mengganti nilai menggunakan replace()
        String nilaiLamaReplace = days.replace(4, "Rabo");
        System.out.println("Setelah replace(4, \"Rabo\"): " + days);
        System.out.println("Nilai lama replace key 4: " + nilaiLamaReplace);

        // Replace pada key yang tidak ada
        days.replace(5, "Kamis");
        System.out.println("Setelah replace(5, \"Kamis\"): " + days);
    }
}
```

Output

```
Isi awal: {1=Minggu, 2=Senin, 3=Selasa}
Setelah put(1, "Ahad"): {1=Ahad, 2=Senin, 3=Selasa}
Nilai lama key 1: Minggu
Setelah put(4, "Rabu"): {1=Ahad, 2=Senin, 3=Selasa, 4=Rabu}
Setelah replace(4, "Rabo"): {1=Ahad, 2=Senin, 3=Selasa, 4=Rabo}
Nilai lama replace key 4: Rabu
Setelah replace(5, "Kamis"): {1=Ahad, 2=Senin, 3=Selasa, 4=Rabo}
```

3.2 Retrieving Data

Retrieving data from a HashMap using `get()` can be compared to **checking in at the station using a booking code**. The staff does not ask for the passenger's position in line or when the ticket was purchased. Instead, they enter the booking code and instantly retrieve the passenger's details. This demonstrates the efficiency of HashMap, where data retrieval depends solely on the key and not on the size of the stored data.



```
import java.util.HashMap;

public class OperasiGet {
    public static void main(String[] args) {

        HashMap<Integer, String> days = new HashMap<>();
        days.put(1, "Ahad");
        days.put(2, "Senin");
        days.put(3, "Selasa");

        // Mengambil nilai berdasarkan key
        String hariKedua = days.get(2);
        System.out.println("Hari kedua: " + hariKedua);

        // Mengambil key yang tidak ada
        String hariKelima = days.get(5);
        System.out.println("Hari kelima: " + hariKelima);
    }
}
```

Output:

```
Hari kedua: Senin
Hari kelima: null
```

3.3 Removing Data

Removing data from a HashMap is similar to **canceling a train ticket or resetting daily reservation records**. When a passenger cancels their booking, only their specific record is removed. At the end of the day, the system may clear all temporary data to prepare for a new schedule. HashMap supports both targeted removal (`remove`) and complete data clearance (`clear`) to manage stored information efficiently.



```
import java.util.HashMap;

public class OperasiRemoveClear {
    public static void main(String[] args) {

        HashMap<Integer, String> days = new HashMap<>();
        days.put(1, "Ahad");
        days.put(2, "Senin");
        days.put(3, "Selasa");
        days.put(7, "Sabtu");

        System.out.println("Isi sebelum remove: " + days);

        // Menghapus satu data
        String hariDihapus = days.remove(1);
        System.out.println("Isi setelah remove(1): " + days);
        System.out.println("Hari yang dihapus: " + hariDihapus);

        // Menghapus semua data
        days.clear();
        System.out.println("Isi setelah clear(): " + days);
    }
}
```

Output:

```
Isi sebelum remove: {1=Ahad, 2=Senin, 3=Selasa, 7=Sabtu}
Isi setelah remove(1): {2=Senin, 3=Selasa, 7=Sabtu}
Hari yang dihapus: Ahad
Isi setelah clear(): {}
```

3.4 Checking Data Existence

Checking whether a key or value exists in a HashMap is like **verifying whether a booking code or passenger name is registered in the system** before proceeding with an operation. For example, before allowing a passenger to board, staff must confirm that the booking code exists. Similarly, checking values may be useful to determine whether a specific passenger is already registered. These checks prevent invalid operations and ensure system reliability.

Methode	Description
<code>containsKey(key)</code>	Check If Key exist
<code>containsValue(value)</code>	Check if value exist



```
import java.util.HashMap;

public class CheckingDataExistence {
    public static void main(String[] args) {

        // Membuat HashMap data penumpang
        HashMap<String, String> dataPenumpang = new HashMap<>();

        // Menambahkan data penumpang
        dataPenumpang.put("KA001", "Budi Santoso");
        dataPenumpang.put("KA002", "Siti Aisyah");
        dataPenumpang.put("KA003", "Andi Pratama");

        // Mengecek keberadaan key (kode booking)
        String kodeCek = "KA002";
        if (dataPenumpang.containsKey(kodeCek)) {
            System.out.println("Kode booking " + kodeCek + " terdaftar.");
        } else {
            System.out.println("Kode booking " + kodeCek + " tidak terdaftar.");
        }

        // Mengecek keberadaan value (nama penumpang)
        String namaCek = "Dewi Lestari";
        if (dataPenumpang.containsValue(namaCek)) {
            System.out.println("Penumpang " + namaCek + " sudah terdaftar.");
        } else {
            System.out.println("Penumpang " + namaCek + " belum terdaftar.");
        }
    }
}
```

3.5 Other Useful HashMap Methods

Table Commonly Used HashMap Methods

Method	Description	Example Usage	Return Value
<code>containsKey(Object key)</code>	Checks whether a specific key exists in the map.	<i>boolean hasTuesday = days.containsKey(3);</i>	true if key exists, otherwise false
		<i>boolean hasSunday = days.containsKey(1);</i>	false
<code>containsValue(Object value)</code>	Checks whether a specific value exists in the map. Note: This operation can be	<i>boolean hasMonday = days.containsValue("Senin");</i>	true



	slow ($O(n)$) because it checks all values.		
		<i>boolean</i> <i>hasWednesday =</i> <i>days.containsValue("R</i> <i>abu");</i>	false
<code>size()</code>	Returns the total number of key-value pairs (entries) in the map.	<i>int totalDays =</i> <i>days.size();</i>	2
<code>isEmpty()</code>	Checks whether the map is empty (contains no entries).	<i>boolean isEmpty =</i> <i>days.isEmpty();</i>	false
		<i>HashMap<String,</i> <i>String> emptyMap =</i> <i>new HashMap<>();</i>	
		<i>boolean checkEmpty</i> <i>=</i> <i>emptyMap.isEmpty();</i>	true



Table Methods for Data Traversal and Duplication

Method	Description	Example Usage	Return Value
<code>keySet()</code>	Returns a <code>Set<K></code> containing all keys in the map. Order is not guaranteed.	<code>Set<Integer> dayKeys = days.keySet();</code>	<code>{2, 3}</code> (order may vary)
<code>values()</code>	Returns a <code>Collection<V></code> containing all values in the map. Duplicate values may exist. Order is not guaranteed.	<code>Collection<String> dayNames = days.values();</code>	<code>["Senin", "Selasa"]</code> (order may vary)
<code>entrySet()</code>	Returns a <code>Set<Map.Entry<K, V>></code> containing all key-value pairs. This is the most efficient way to iterate through a map.	<code>Set<Map.Entry<Integer, String>> entries = days.entrySet();</code>	<code>{2=Senin, 3=Selasa}</code> (order may vary)
<code>clone()</code>	Creates a shallow copy of the <code>HashMap</code> . Changes to mutable objects inside the copy will affect the original map.	<code>HashMap<Integer, String> copyDays = (HashMap<Integer, String>) days.clone();</code>	A new <code>HashMap</code> object with the same content



Implementation code below:

```
import java.util.*;

public class MethodLain {
    public static void main(String[] args) {

        HashMap<Integer, String> days = new HashMap<>();
        days.put(2, "Senin");
        days.put(3, "Selasa");
        days.put(4, "Rabu");

        System.out.println("Ukuran map: " + days.size());
        System.out.println("Apakah key 4 ada? " + days.containsKey(4));
        System.out.println("Apakah value 'Jumat' ada? " +
        days.containsValue("Jumat"));
        System.out.println("Apakah map kosong? " + days.isEmpty());

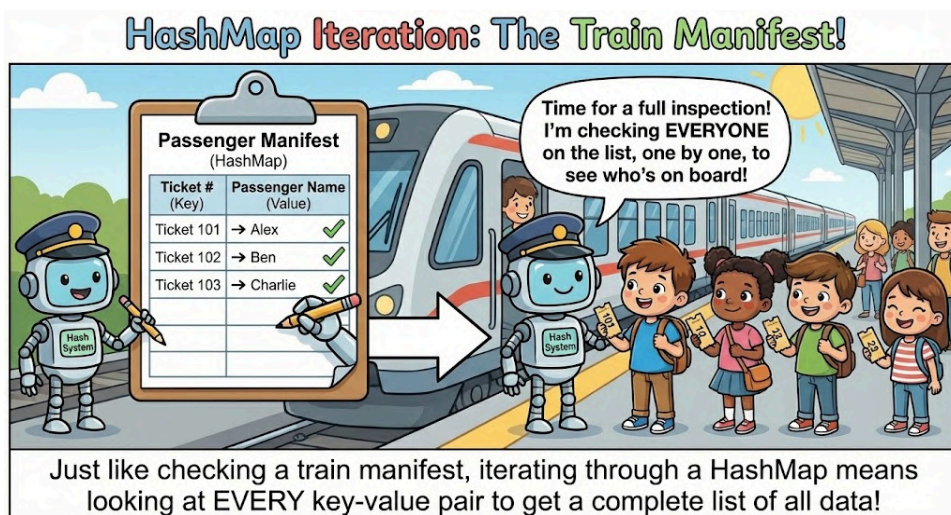
        Set<Integer> keys = days.keySet();
        System.out.println("Semua Key: " + keys);

        Collection<String> values = days.values();
        System.out.println("Semua Value: " + values);

        Set<Map.Entry<Integer, String>> entries = days.entrySet();
        System.out.println("Semua Entry: " + entries);
    }
}
```

4. Iteration & Data Traversal

Iterating through a HashMap is comparable to **printing a complete passenger manifest for a train**. Instead of looking up one booking at a time, staff may need a full list of passengers for inspection or reporting purposes. By traversing all key value pairs, the system can display every registered booking and its corresponding passenger details. This process does not depend on order, only completeness.



4.1 Iteration Using entrySet()

```

import java.util.HashMap;
import java.util.Map;

public class IterasiHashMap {
    public static void main(String[] args) {

        // Membuat HashMap data penumpang
        HashMap<String, String> dataPenumpang = new HashMap<>();

        // Menambahkan data penumpang
        dataPenumpang.put("KA001", "Titan Cahyadi");
        dataPenumpang.put("KA002", "Budi Sukatmono");
        dataPenumpang.put("KA003", "Dika Cellow");
        dataPenumpang.put("KA004", "Fatih Destroyer");

        // Menampilkan manifest penumpang menggunakan entrySet()
        System.out.println("≡ MANIFEST PENUMPANG KERETA API ≡");

        for (Map.Entry<String, String> entry : dataPenumpang.entrySet()) {
            System.out.println(
                "Kode Booking : " + entry.getKey() +
                " | Nama Penumpang : " + entry.getValue()
            );
        }
    }
}

```

Output:

```

≡ MANIFEST PENUMPANG KERETA API ≡
Kode Booking : KA004 | Nama Penumpang : Fatih Destroyer
Kode Booking : KA003 | Nama Penumpang : Dika Cellow
Kode Booking : KA002 | Nama Penumpang : Budi Sukatmono
Kode Booking : KA001 | Nama Penumpang : Titan Cahyadi

```



CODELAB

Exercise

In this exercise, students will create a simple Java program to store, display, search, and delete station officer contact data using the HashMap data structure.

Each contact has:

- **Officer Name** as Key (*String*)
- **Phone Number** as Value (*String*)

```
import java.util.HashMap;
import java.util.Map;

public class KontakStasiun {
    public static void main(String[] args) {

        // TODO1: Inisialisasi HashMap untuk menyimpan kontak
        HashMap<String, String> kontakStasiun = _____;
        // TODO: Tambahkan data kontak menggunakan method put()
        kontakStasiun._____("FATIH - Kepala Stasiun", "081234567890");

        kontakStasiun._____("WIRA - Customer Service", "_____");

        _____._____("EGA - Keamanan", "081255555555");

        // TODO2: Tampilkan seluruh isi kontak menggunakan entrySet()
        System.out.println("≡ Daftar Kontak Stasiun ≡");

        for (Map.Entry<String, String> entry : kontakStasiun._____()) {

            System.out.println(
                "Nama: " + entry._____() +
                " | Telepon: " + entry._____()
            );
        }
    }
}
```



```

// TODO: Cari nomor telepon berdasarkan nama menggunakan get()
String namaDicari = "WIRA - Customer Service";
String nomorTelepon = kontakStasiun._____(namaDicari);
if (nomorTelepon != null) {
    System.out.println(
        "\nNomor telepon " + namaDicari + ": " + nomorTelepon
    );
} else {
    System.out.println(
        "\nKontak " + namaDicari + " tidak ditemukan."
    );
}

// TODO: Cek apakah kontak ada menggunakan containsKey()
String namaPengecekan = "Keamanan";
if (kontakStasiun._____(namaPengecekan)) {
    System.out.println(
        "Kontak " + namaPengecekan + " tersedia di sistem."
    );
} else {
    System.out.println(
        "Kontak " + namaPengecekan + " tidak tersedia."
    );
}

// TODO: Hapus data kontak menggunakan remove()
String namaHapus = "FATIH - Kepala Stasiun";
kontakStasiun._____(namaHapus);
System.out.println(
    "\nIsi kontak setelah menghapus " + namaHapus + ":"
);
System.out.println(kontakStasiun);
}
}

```



Output Program :

You can change their name and job to suit your creativity

```
Nama: FATIH - Kepala Stasiun | Telepon: 081234567890
Nama: EGA - Keamanan | Telepon: 081255555555
Nama: WIRA - Customer Service | Telepon: 081298765432

Nomor telepon WIRA - Customer Service: 081298765432
Kontak Keamanan tidak tersedia.

Isi kontak setelah menghapus FATIH - Kepala Stasiun:
{EGA - Keamanan=081255555555, WIRA - Customer Service=081298765432}
```

This exercise aims to familiarise students with basic HashMap operations before moving on to larger case studies in the DEMO session.

Explanation :

- HashMap<String,String> is used to store pairs of employee names and telephone numbers.
- Method `put()` is used to add data.
- Method `entrySet()` is used to display the entire contents of the HashMap.
- Method `get()` method is used to search for data based on the key.
- Method `containsKey()` is used to validate the existence of data.
- Method `remove()` method is used to delete specific data.



DEMO

TASK 1

In this DEMO, students are required to build a **console-based Java application** that simulates a **Railway Inventory and Passenger Management System**. This system is a continuation of the previous modules and uses **HashMap as the core data structure** for fast data storage and retrieval. The program **must** provide the following menu-based features:

1. **Add New Inventory Item**
 - Input item name
 - Input initial stock
 - Input price
 - Store data using `put ()`
2. **View Inventory List**
 - Display all items
 - Show item name, stock, and price
 - Use iteration with `entrySet ()` or `keySet ()`
3. **Search Inventory Item**
 - Search item by name
 - If found, display stock and price
 - If not found, show error message
4. **Increase Item Stock**
 - Add stock to an existing item
 - Validate item existence using `containsKey ()`
5. **Remove Inventory Item**
 - Remove item by name using `remove ()`
6. **Update Item Price**
 - Modify item price using `put ()` or `replace ()`

```
Railway Inventory System
```

```
1. Add Item
2. View Inventory
3. Search Item
4. Add Stock
5. Remove Item
6. Update Price
7. Exit
```

```
Choose menu: 1
```

```
  Add New Item
```

```
Enter item name: Pocari Sweat
```

```
Enter initial stock: 60
```

```
Enter price: 9000
```

```
Item added successfully!
```

```
Railway Inventory System
```

```
1. Add Item
2. View Inventory
3. Search Item
4. Add Stock
5. Remove Item
6. Update Price
7. Exit
```

```
Choose menu: 2
```

```
Inventory List
```

```
1. Pocari Sweat | Stock: 60 | Price: Rp 9,000
Total items: 1
```



Railway Inventory System

1. Add Item
2. View Inventory
3. Search Item
4. Add Stock
5. Remove Item
6. Update Price
7. Exit

Choose menu: 3

Search Item

Enter item name to search: Pocari Sweat

Item Found!

Name : Pocari Sweat

Stock : 60

Price : Rp 9,000

Railway Inventory System

1. Add Item
2. View Inventory
3. Search Item
4. Add Stock
5. Remove Item
6. Update Price
7. Exit

Choose menu: 4

Add Stock

Enter item name: Pocari Sweat

Enter stock to add: 50

Stock updated successfully!

Pocari Sweat: 60 - 110

Railway Inventory System

1. Add Item
2. View Inventory
3. Search Item
4. Add Stock
5. Remove Item
6. Update Price
7. Exit

Choose menu: 5

Remove Item

Enter item name to remove: Pocari Sweat

Item "Pocari Sweat" removed successfully!
(Stock: 110, Price: Rp 9,000)

Railway Inventory System

1. Add Item
2. View Inventory
3. Search Item
4. Add Stock
5. Remove Item
6. Update Price
7. Exit

Choose menu: 6

Update Price

Enter item name: Glass

Current price: Rp 50,000

Enter new price: 40000

Stock updated successfully!

Glass: Rp 50,000 - Rp 40,000

The output format is flexible. Students may improve UI clarity.



TASK 2

Extend **TASK 1** by adding **user authentication features**. After successfully completing **Task 1 (Railway Inventory Management System)**, students are required to **extend the existing program** by adding a **User Management System**. This feature simulates how a railway inventory system controls access using **registration, login, and session based operations**. The implementation **must reuse and integrate** the inventory system from Task 1.

1. **Setup Authentication Data Structures**
 - a. Initialize users (`HashMap<String, String>`) for login validation (**Key: Username, Value: Password**).
 - b. Initialize `userDetails` (`HashMap<String, ArrayList<String>>`) to store personal info (**Key: Username, Value: [Full Name, Address]**).
 - c. Add a `loggedInUser` (`String`) variable to track the active session (set to `null` by default).
2. **Implement Registration Feature (`register()`)**
 - a. Input: Prompt user for Username, Password, Full Name, and Address.
 - b. Validation:
 - Username must be unique (not already in `users`).
 - Username must contain the character "@".
 - Passwords must be at least 8 characters long.
 - c. Action: If valid, store credentials in `users` and personal info in `userDetails`, then print **"Registration Successful!"**. Otherwise, print **"Registration Failed!"**.
3. **Implement Login Feature (`login()`)**
 - a. Input: Prompt user for **Username** and **Password**.
 - b. Action: Validate against the `users` `HashMap`. If matched, update `loggedInUser` with the username and print **"Login Successful!"** followed by a personalized welcome message using the Full Name and Address from `userDetails`. If mismatched, print **"Login Failed!"**.
4. **Implement Logout Feature(`logout()`)**
 - a. Action: Reset `loggedInUser` back to `null`, print a confirmation message, and restrict the user back to the Pre-Login Menu.
5. **Implement View Inventory Report (`viewInventoryReport()`)**
 - a. Condition: Strictly accessible only when a user is logged in (`loggedInUser != null`).
 - b. Action: Display a full list of inventory items (Name, Stock, Price). At the bottom of the report, print the currently logged-in user's name.
6. **Apply Dynamic Menu (Access Control)**
 - a. Modify the main loop to handle two different access states:
 - Pre-Login Menu: User can only access 1. Register, 2. Login, and 3. Exit.
 - Post-Login Menu: User can access All Task 1 Features + View Inventory Report + Logout.



Expected Output

The output should at least look like the example below, but it would be even better if it were enhanced creatively.

```
Welcome to the Railway Inventory System
Please select a menu:
1. Register
2. Login
3. Exit
Choose menu: 1

Username: AlJ@hadiun
Password:
*****
Full Name: Al Jahidun dul Zaelani
Address: Tegalgondo 38 Malang
Registration Successful!
```

1. Register

```
Username: AlJ@hadiun
Password:
*****
Login Successful!
Welcome, Al Jahidun dul Zaelani (Tegalgondo 38 Malang)

Please select a menu:
1. Add New Item
2. View Inventory List
3. Search Item
4. Add Item Stock
5. Remove Item
6. Update Item Price
7. View Inventory Report
8. Logout
Choose menu: 7
```

2. Pre-Login Menu

```
Inventory Report:
- Mineral Water Pack: Stock = 50, Price = 60000.0
- Train Meal Box: Stock = 100, Price = 75000.0

Logged-in User: Al Jahidun dul Zaelani
```

3. PostLogin Menu

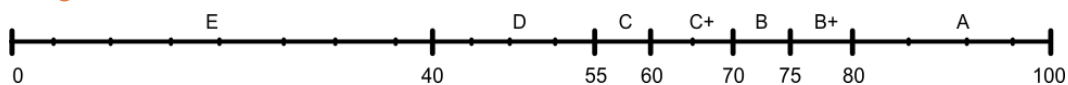


Assesment

Grading Rubric

Assessment Aspect	Points (100%)
Codelab	Total 20 %
Initial Familiarisation with Hashmap	10%
Successfully run and understand the simple contact Codelab code.	10%
Demo	Total 80%
Task 1	40%
Implement Hashmap	10%
Implement Data Iterations	5%
Implement Collisions	5%
Answer Questions	20%
Task 2	40%
Implement Hashmap	15%
Running Smoothly	5%
Answer Questions	20%

Grading Scale



- A = (81 - 100) → Sepuh
- B+ = (75 - 80) → Very Good
- B = (70 - 74) → Good
- C+ = (60 - 69) → Fairly Good
- C = (55 - 59) → Fair
- D = (41 - 54) → Poor
- E = (0 - 40) → Bro really...

